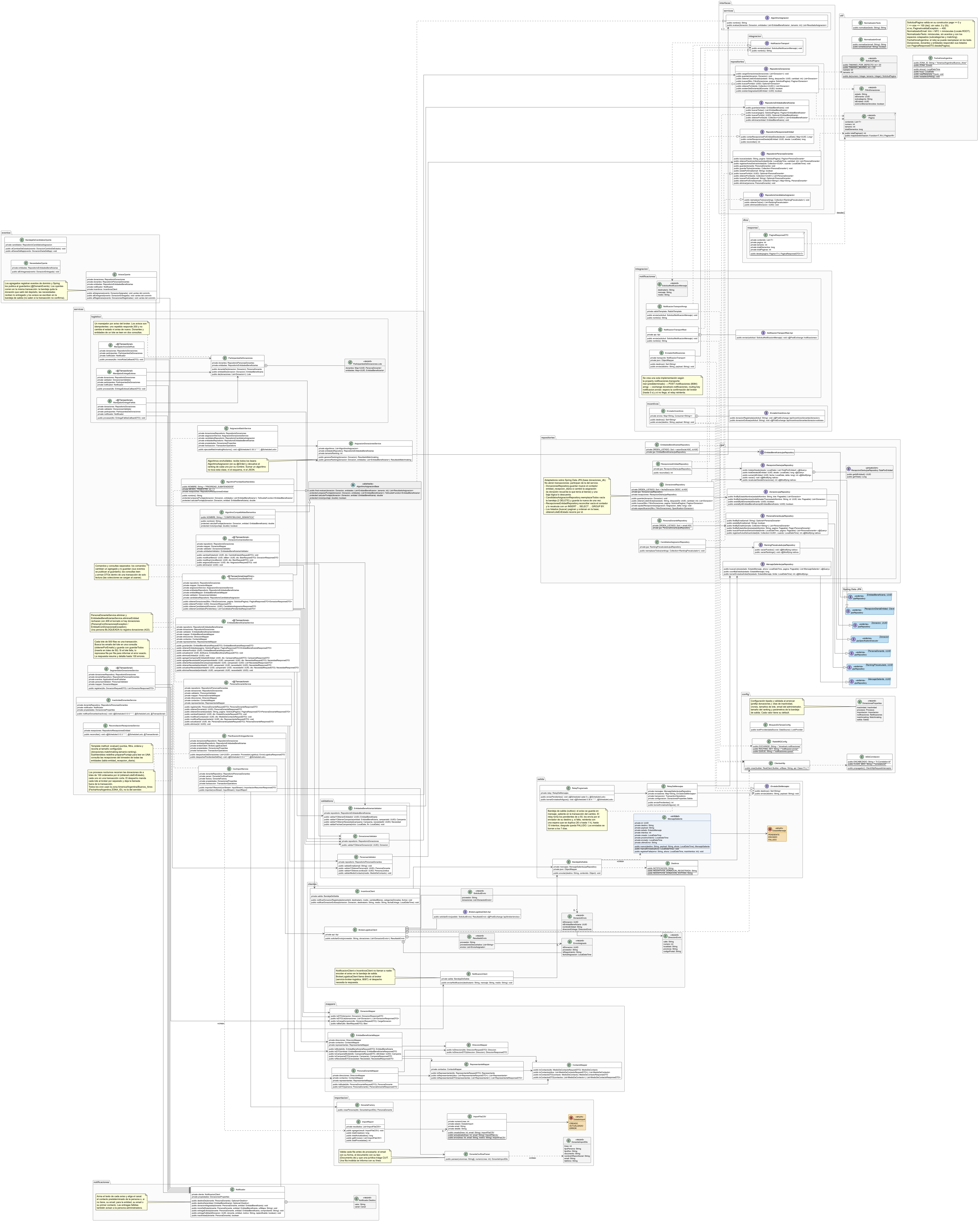

interfaces
services
AlgoritmoAsignacion
integracion
NotificacionTransport
repositorios
RepositorioDonaciones
RepositorioEntidadesBeneficiarias
RepositorioRecepcionesEntidad
RepositorioPersonasDonantes
RepositorioCandidatosAsignacion
util
NormalizadorTexto
NormalizadorEmail
«record» SolicitudPagina
FechaHoraArgentina
public ZONA_ID: String = "America/Argentina/Buenos_Aires"
public ZONA: ZoneId
public ahora(): LocalDateTime
public hoy(): LocalDate
public usarReloj(reloj: Clock): void
public restablecerReloj(): void
«record» FiltroDonaciones
«record» Pagina
dtos
response
PaginaResponseDTO
eventos
BandejaDeCandidatosOyente
private candidatos: RepositorioCandidatosAsignacion
public alCambiarDeEstado(evento: DonacionCambioDeEstado): void
public alDarseDeBaja(evento: DonacionDadaDeBaja): void
NecesidadesOyente
private entidades: RepositorioEntidadesBeneficiarias
public alEntregarse(evento: DonacionEntregada): void
AvisosOyente
private donaciones: RepositorioDonaciones
private donantes: RepositorioPersonasDonantes
private entidades: RepositorioEntidadesBeneficiarias
private notificador: Notificador
private incentivos: IncentivosClient
public alAsignarse(evento: DonacionAsignada): void «antes del commit»
public alEntregarse(evento: DonacionEntregada): void «antes del commit»
public alRegistrarse(evento: DonacionesRegistradas): void «antes del commit»
services
logistica
«@Transactional» ManejadorInicioDeRuta
private donaciones: RepositorioDonaciones
private participantes: ParticipantesDeDonaciones
private notificador: Notificador
public procesar(dto: InicioRutaCallbackDTO): void
«@Transactional» ManejadorEntregaExitosa
private donaciones: RepositorioDonaciones
private validador: DonacionesValidator
private participantes: ParticipantesDeDonaciones
private notificador: Notificador
public procesar(dto: EntregaExitosaCallbackDTO): void
«@Transactional» ManejadorEntregaFallida
private donaciones: RepositorioDonaciones
private validador: DonacionesValidator
private participantes: ParticipantesDeDonaciones
private notificador: Notificador
public procesar(dto: EntregaFallidaCallbackDTO): void
ParticipantesDeDonaciones
private donantes: RepositorioPersonasDonantes
private entidades: RepositorioEntidadesBeneficiarias
public donanteDe(donacion: Donacion): PersonaDonante
public entidadDe(donacion: Donacion): EntidadBeneficiaria
public de(donaciones: List<Donacion>): Lote
«record» ParticipantesDeDonaciones.Lote
AsignacionBatchService
private donacionesRepositorio: RepositorioDonaciones
private asignacionService: AsignacionDonacionesService
private candidatosRepositorio: RepositorioCandidatosAsignacion
private entidadesRepositorio: RepositorioEntidadesBeneficiarias
private propiedades: DonacionesProperties
private transaccion: TransactionOperations
public ejecutarMatchmakingNocturno(): void «@Scheduled 0 30 3 * * *, @SchedulerLock»
AsignacionDonacionesService
«abstract» AlgoritmoAsignacionBase
AlgoritmoPrioridadSubAtendidos
public NOMBRE: String = "PRIORIDAD_SUBATENDIDOS"
private MESES_TRIMESTRE: int = 3
private recepciones: RepositorioRecepcionesEntidad
public nombre(): String
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
AlgoritmoCompatibilidadSemantica
public NOMBRE: String = "COMPATIBILIDAD_SEMANTICA"
public nombre(): String
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
protected incluir(puntaje: double): boolean
«@Transactional» DonacionComandoService
«@Transactional(readOnly)» DonacionConsultaService
«@Transactional» EntidadesBeneficiariasService
«@Transactional» SegmentadorDonacionesService
private donacionesRepository: RepositorioDonaciones
private donanteRepository: RepositorioPersonasDonantes
private eventos: ApplicationEventPublisher
private personasValidator: PersonasValidator
private mapper: DonacionMapper
public registrar(dto: DonacionRequestDTO): List<DonacionResponseDTO>
«@Transactional» PersonaDonanteService
InactividadDonantesService
private donanteRepository: RepositorioPersonasDonantes
private notificador: Notificador
private propiedades: DonacionesProperties
public notificarDonantesInactivos(): void «@Scheduled 0 0 0 * * *, @SchedulerLock, @Transactional»
ReconciliacionRecepcionesService
private recepciones: RepositorioRecepcionesEntidad
public reconciliar(): void «@Scheduled 0 0 3 * * *, @SchedulerLock, @Transactional»
PlanificacionEntregasService
CsvImportService
validations
EntidadesBeneficiariasValidator
DonacionesValidator
PersonasValidator
clientes
IncentivosClient
«record» IncentivosClient...
BrokerLogisticaClient.Api
BrokerLogisticaClient
«record» DonacionEnvio
«record» ResultadoEnvio
«record» EnvioAsignado
«record» DireccionEnvio
NotificacionClient
«crea»
mappers
DonacionMapper
public toDTO(donacion: Donacion): DonacionResponseDTO
public toDTOList(donaciones: List<Donacion>): List<DonacionResponseDTO>
public toCargaDonacion(dto: DonacionRequestDTO): CargaDonacion
public toBien(dto: BienRequestDTO): Bien
EntidadBeneficiariaMapper
private direcciones: DireccionMapper
private contactos: ContactoMapper
private representantes: RepresentanteMapper
public toModel(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
public toDTO(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
public toCampaniaModel(dto: CampaniaRequestDTO, idEntidad: UUID): Campania
public toCampaniaDTO(campania: Campania): CampaniaResponseDTO
public toNecesidadDTO(necesidad: Necesidad): NecesidadResponseDTO
PersonaDonanteMapper
private direcciones: DireccionMapper
private contactos: ContactoMapper
private representantes: RepresentanteMapper
public toModel(dto: PersonaDonanteRequestDTO): PersonaDonante
public toDTO(persona: PersonaDonante): PersonaDonanteResponseDTO
DireccionMapper
RepresentanteMapper
ContactoMapper
importacion
DonanteFactory
public crearPersona(dto: DonanteImportDto): PersonaDonante
ImportReport
private resultados: List<ImportFilaCSV>
public agregar(result: ImportFilaCSV): void
public totalCreados(): long
public totalActualizados(): long
public getErrores(): List<ImportFilaCSV>
public totalProcesadas(): int
ImportFilaCSV
Estado
CREADO
ACTUALIZADO
ERROR
DonanteCsvRowParser
«record» DonanteImportDto
«crea»
notificaciones
Notificador
«record» Notificador.Destino
valor: String
canal: Canal
integracion
notificaciones
«record» SolicitudNotificacionMensaje
NotificacionTransportAmqp
private rabbitTemplate: RabbitTemplate
public enviar(solicitud: SolicitudNotificacionMensaje): void
public nombre(): String
NotificacionTransportRest
private api: Api
public enviar(solicitud: SolicitudNotificacionMensaje): void
public nombre(): String
NotificacionTransportRest.Api
EnviadorNotificaciones
private transporte: NotificacionTransport
private json: ObjectMapper
public destinos(): Set<String>
public enviar(destino: String, payload: String): void
incentivos
EnviadorIncentivos
private envios: Map<String, Consumer<String>>
public destinos(): Set<String>
public enviar(destino: String, payload: String): void
EnviadorIncentivos.Api
repositories
EntidadesBeneficiariasRepository
private ORDEN_LISTADO: Sort = razonSocial ASC, id ASC
private jpa: EntidadBeneficiariaJpaRepository
RecepcionesEntidadRepository
private jpa: RecepcionDiariaJpaRepository
public reconciliar(): int
DonacionesRepository
private ORDEN_LISTADO: Sort = fechaDonacion DESC, id ASC
private jpa: DonacionJpaRepository
private recepciones: RecepcionDiariaJpaRepository
public guardar(donacion: Donacion): void
public obtenerLoteEnEstado(estado: String, despuesDe: UUID, cantidad: int): List<Donacion>
public buscar(filtro: FiltroDonaciones, pagina: SolicitudPagina): Pagina<Donacion>
private ajustarRecepciones(asignacion: Asignacion, delta: long): void
private especificacion(filtro: FiltroDonaciones): Specification<Donacion>
PersonaDonanteRepository
private ORDEN_LISTADO: Sort = email ASC
private jpa: PersonaDonanteJpaRepository
CandidatosAsignacionRepository
private jpa: RankingPrecalculadoJpaRepository
public reemplazarTodos(rankings: Collection<RankingPrecalculado>): void
jpa
EntidadBeneficiariaJpaRepository
RecepcionDiariaJpaRepository
«projection» RecepcionDiariaJpaRepository.TotalPorEntidad
DonacionJpaRepository
PersonaDonanteJpaRepository
RankingPrecalculadoJpaRepository
MensajeSalienteJpaRepository
Spring Data JPA
«external» JpaRepository
«external» JpaSpecificationExecutor
config
«record» DonacionesProperties
BloqueoDeTareasConfig
public lockProvider(dataSource: DataSource): LockProvider
RabbitMQConfig
public EXCHANGE: String = "donatrack.notificaciones"
public ROUTING_KEY: String = "notificacion.enviar"
public QUEUE: String = "notificaciones.queue"
ClientesHttp
public crear(builder: RestClient.Builder, urlBase: String, api: Class<T>): T
IdDeCorrelacion
salida
RelayProgramado
private relay: RelayDeMensajes
public enviarPendientes(): void «@Scheduled cada 5 s, @SchedulerLock»
public borrarEnviadosAntiguos(): void «@Scheduled 0 30 4 * * *, @SchedulerLock»
RelayDeMensajes
EnviadorDeMensajes
«entity» MensajeSaliente
«enum» EstadoMensaje
PENDIENTE
ENVIADO
FALLIDO
BandejaDeSalida
private mensajes: MensajeSalienteJpaRepository
private json: ObjectMapper
public encolar(destino: String, contenido: Object): void
Destinos
«crea»

SolicitudPagina valida en su constructor page >= 0 y
1 <= size <= 100 (de(): sin valor, 0 y 20);
si no, PaginaInvalidaException → 400.
NormalizadorEmail: trim + NFC + minúsculas (Locale.ROOT).
NormalizadorTexto: minúsculas, sin acentos y con los
espacios colapsados (subcategorías y matching).
FechaHoraArgentina: el reloj se puede reemplazar en los tests.
Donaciones, donantes y entidades responden sus listados
con PaginaResponseDTO.desde(Pagina).

Los agregados registran eventos de dominio y Spring
los publica al guardarlos (@DomainEvents). Los oyentes
corren en la misma transacción: la bandeja quita la
donación que salió del depósito, las necesidades
reciben lo entregado y los avisos se escriben en la
bandeja de salida (no salen si la transacción no confirma).

Un manejador por aviso del broker. Los avisos son
idempotentes: uno repetido responde 200 y no
cambia el estado ni avisa de nuevo. Donantes y
entidades de un lote se leen en dos consultas.

Algoritmos enchufables: recibe todos los beans
AlgoritmoAsignacion (en su @Order) y devuelve el
ranking de cada uno por su nombre. Sumar un algoritmo
no toca esta clase, ni el esquema, ni el JSON.

Comandos y consultas separados: los comandos
cambian un agregado y lo guardan (sus eventos
se publican al guardarlo); las consultas leen
y arman DTOs dentro de una transacción de solo
lectura (las colecciones se cargan al usarse).

PersonaDonanteService.eliminar y
EntidadesBeneficiariasService.eliminarEntidad
rechazan con 409 el borrado si hay donaciones
(PersonaConDonacionesException /
EntidadConDonacionesException).
Una persona BLOQUEADA no registra donaciones (422).

Cada lote de 500 filas es una transacción.
Busca los emails del lote en una consulta
(obtenerPorEmails) y guarda con guardarTodos
(inserts en lotes de 50). Si el lote falla, lo
reprocesa fila por fila para informar el error exacto.
La respuesta resume y detalla hasta 100 errores.

Template method: evaluar() puntúa, filtra, ordena y
recorta al tamaño configurado
(donaciones.matchmaking.tamanio-ranking).
SubAtendidos redefine prepararPuntaje para leer en UNA
consulta las recepciones del trimestre de todas las
entidades (tabla entidad_recepcion_diaria).

Los procesos nocturnos recorren las donaciones de a
lotes de 100 ordenados por id (obtenerLoteEnEstado),
cada uno en una transacción corta. El despacho manda
cada lote al broker por separado y deja la llamada
fuera de la transacción.
Todos los cron usan la zona America/Argentina/Buenos_Aires
(FechaHoraArgentina.ZONA_ID), no la del servidor.

Se crea una sola implementación según
la property notificaciones.transporte:
rest (predeterminado) → POST /notificaciones (8084)
amqp → exchange donatrack.notificaciones, routing key
notificacion.enviar; espera la confirmación del broker
(hasta 5 s) y si no llega, el relay reintenta.

Adaptadores sobre Spring Data JPA (base donaciones_db).
No abren transacciones: participan de la del service.
- DonacionesRepository.guardar mueve el contador
  entidad_recepcion_diaria si cambió la asignación
  (la donación recuerda la que tenía al leerse) y una
  baja lógica lo descuenta.
- CandidatosAsignacionRepository.reemplazarTodos vacía
  la bandeja (2 DELETE) y guarda la nueva de una vez.
- RecepcionesEntidadRepository.reconciliar vacía el contador
  y lo recalcula con un INSERT ... SELECT ... GROUP BY.
- Los listados (buscar) paginan y ordenan en la base;
  obtenerLoteEnEstado recorre por id.

Configuración tipada y validada al arrancar
(prefijo donaciones.): días de inactividad,
crones, tamaños de lote, email del administrador,
tamaño del ranking y parámetros de la bandeja
de salida. Cada valor tiene su default.

Bandeja de salida (outbox): el aviso se guarda en
mensaje_saliente en la transacción del cambio. El
relay toma los pendientes de a 50, los envía por el
enviador de su destino y, si falla, reintenta con
una espera que se duplica (30 s hasta 1 h), hasta
10 intentos; después queda FALLIDO. Los enviados se
borran a los 7 días.

NotificacionClient e IncentivosClient no llaman a nadie:
encolan el aviso en la bandeja de salida.
BrokerLogisticaClient llama directo al broker
(servicio-broker-logistica, 8087): el despacho
necesita la respuesta.

Valida cada fila antes de procesarla: el email
con su forma, el documento con su tipo
(Documento.de) y que una jurídica traiga CUIT.
Una fila inválida se informa con su línea.

Arma el texto de cada aviso y elige el canal:
el contacto predeterminado de la persona o, si
no tiene, su email; para la entidad, su email o
su primer contacto. Las entregas fallidas
también avisan a la persona administradora.